

MDD

MDD — Markdown with Diagrams

テキストから図を生成する軽量プリプロセッサ

図の生成はすごく便利

認知負荷を下げるために、テキストだけでは伝わりにくい情報を図にすると便利。



しかし、AIで図の生成は課題が多い

AIが思ったとおりに図を作ってくれない



悩み1

AIが1つの図を作るだけで数分かかる



悩み2

AIが作った図はバージョン管理が難しい



悩み3

Q AIに作図させるのはなぜこんなにも難しいのか？

A AIが情報を扱う上で、GUIはノイズでしかないから。
図はAIにとって不要な情報の塊でしかない

Q ではどうすればAIに図をうまく扱えるか？

A AIにはデータ構造だけを管理させて
人間のためにいい感じに図にするツール(= プログラム)を作ったらい

ということで作りました。

 [ppdx999](#) / [mdd](#)

Markdown with Diagrams — テキストから図を生成する軽量プリプロセッサ

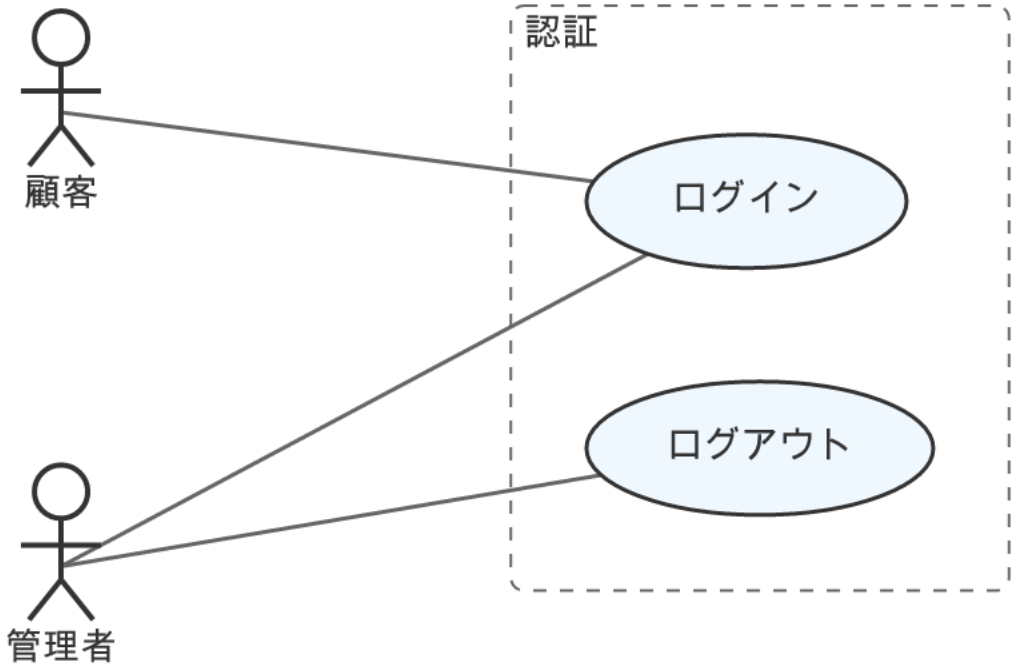
 Rust

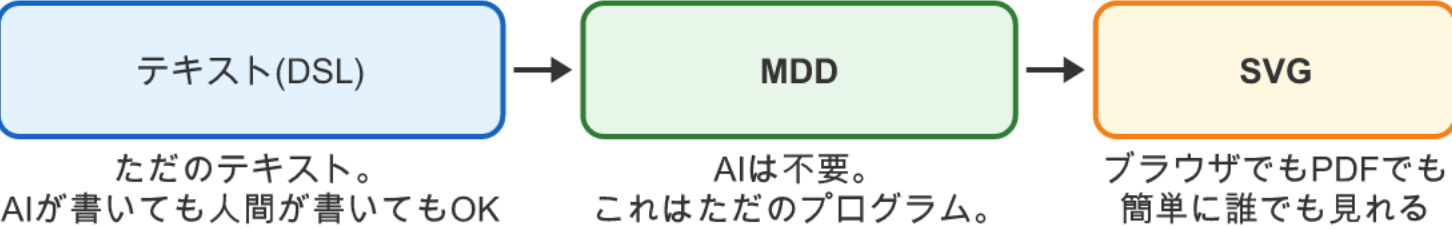
 MIT

テキストからいい感じのSVGを生成するプログラム
例)

```
1 actor 顧客
2 actor 管理者
3
4 package "認証" {
5   usecase ログイン
6   usecase ログアウト
7 }
8
9 顧客 -> ログイン
10 管理者 -> ログイン
11 管理者 -> ログアウト
```

これをmddが変換すると...





余談

このテキストから図を作る技術は1981年に出させた論文の技術が元になっています。詳しく知りたい人はsugiyama法で検索！

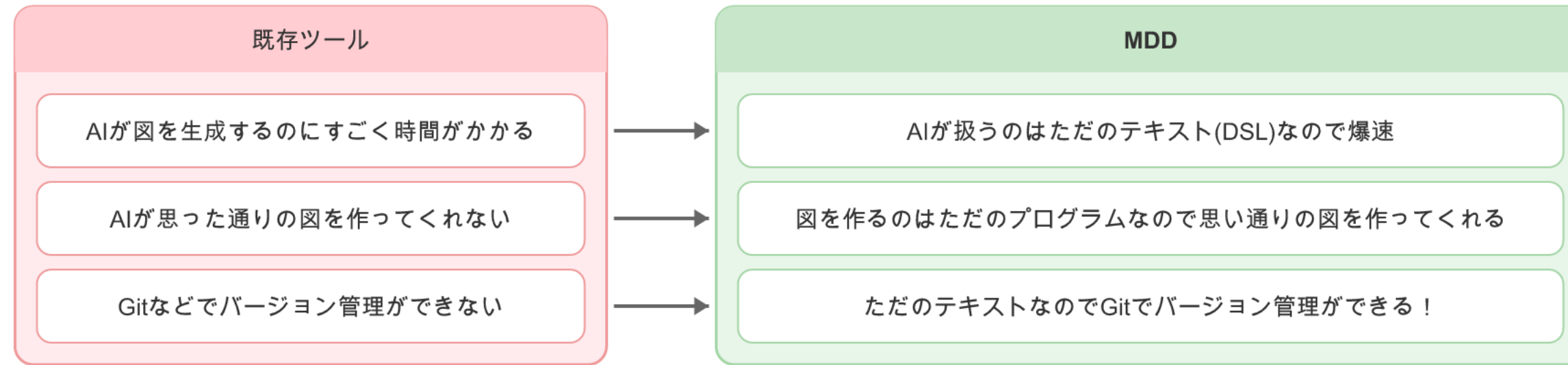
この仕組みがなぜ良いのか？

1 AIはデータの構造だけに集中できるので爆速

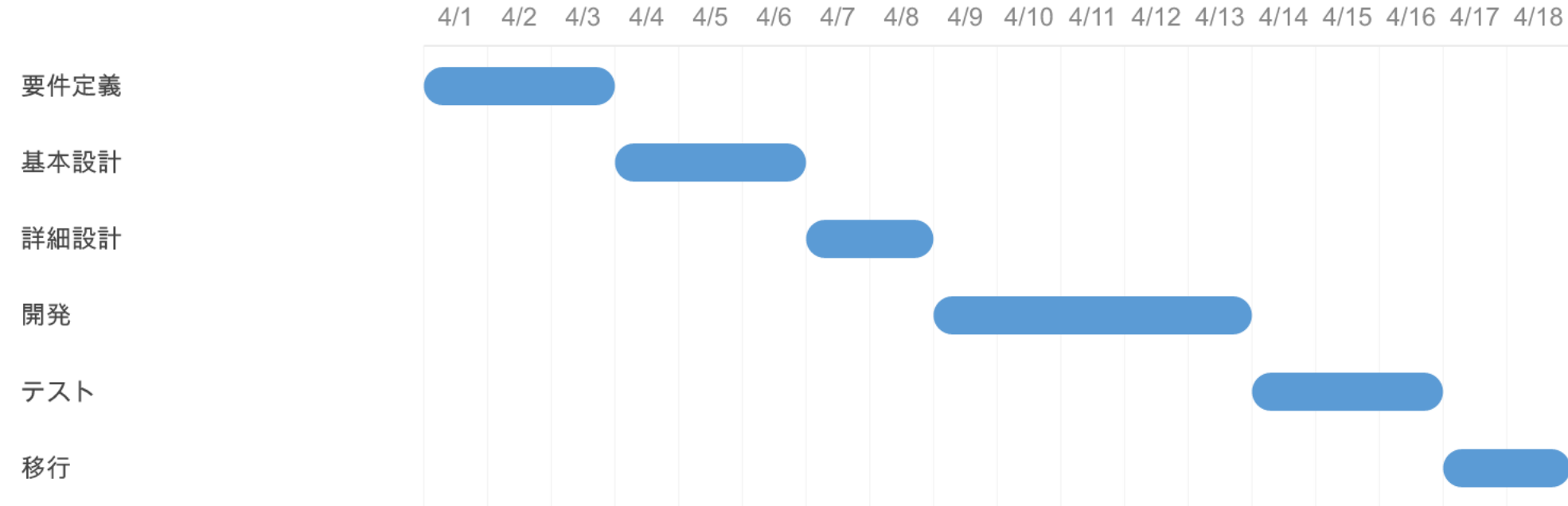
2 レンダリングはただのプログラムがやるので爆速

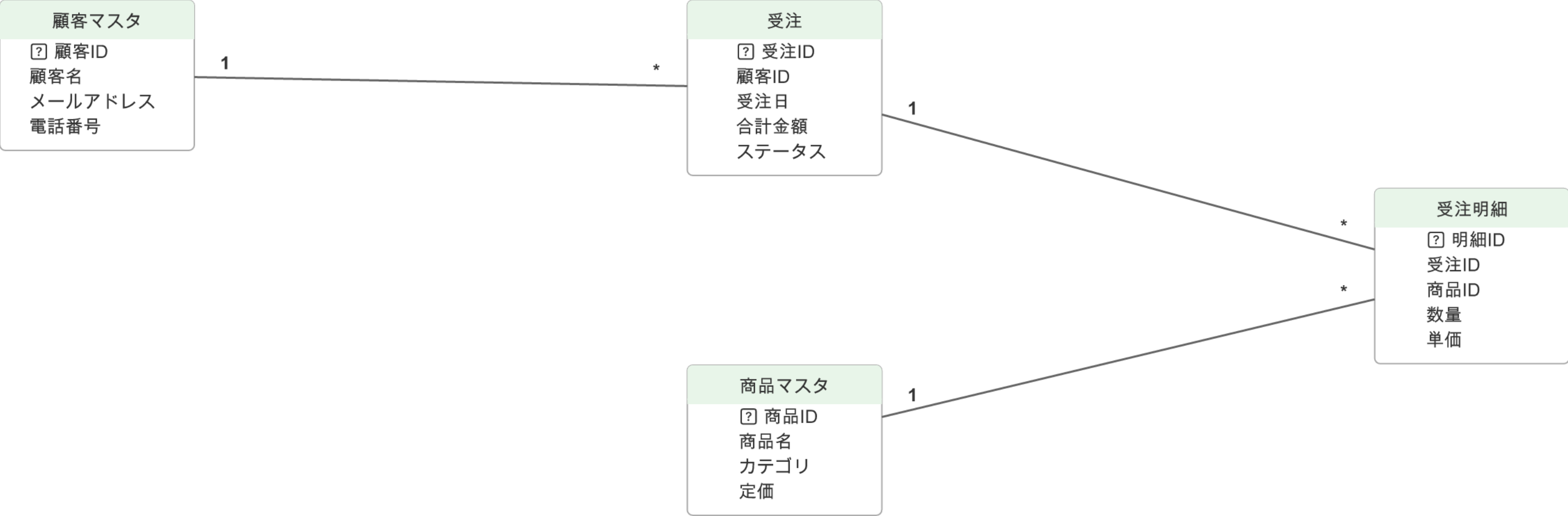
3 全ての元データがただのテキストになる

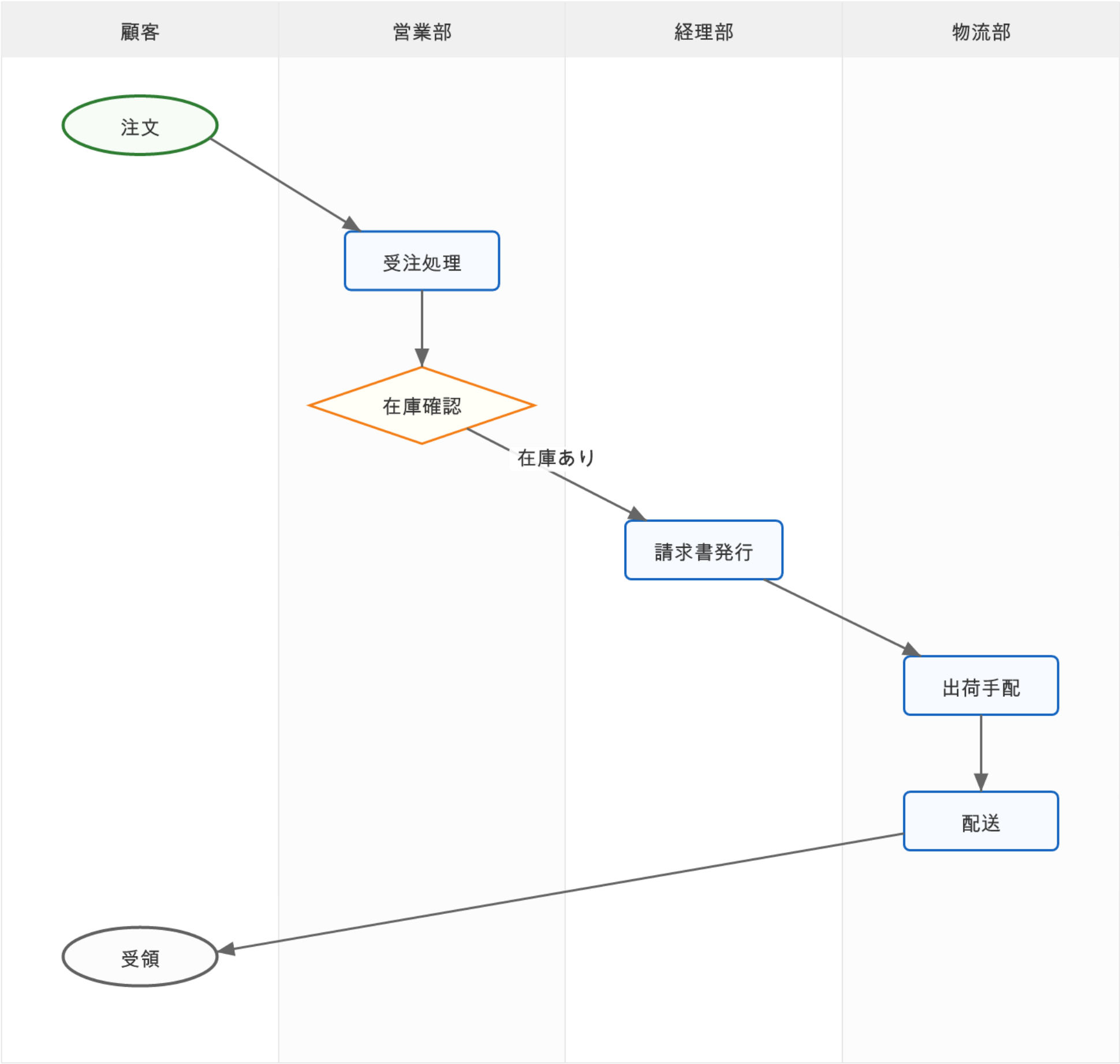
これの何が嬉しい？

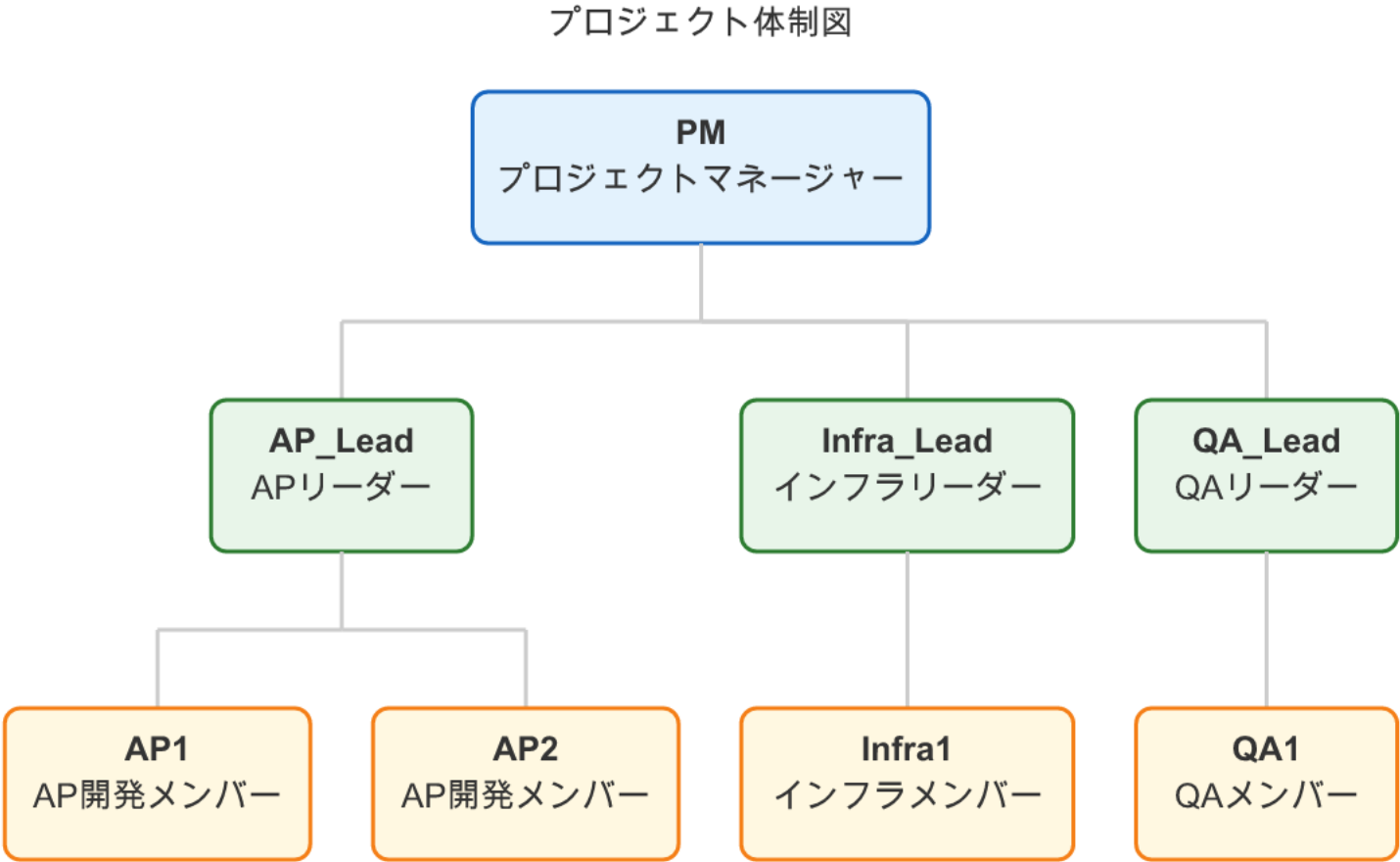


基幹システム刷新PJ









フレームワーク比較

Spring Boot

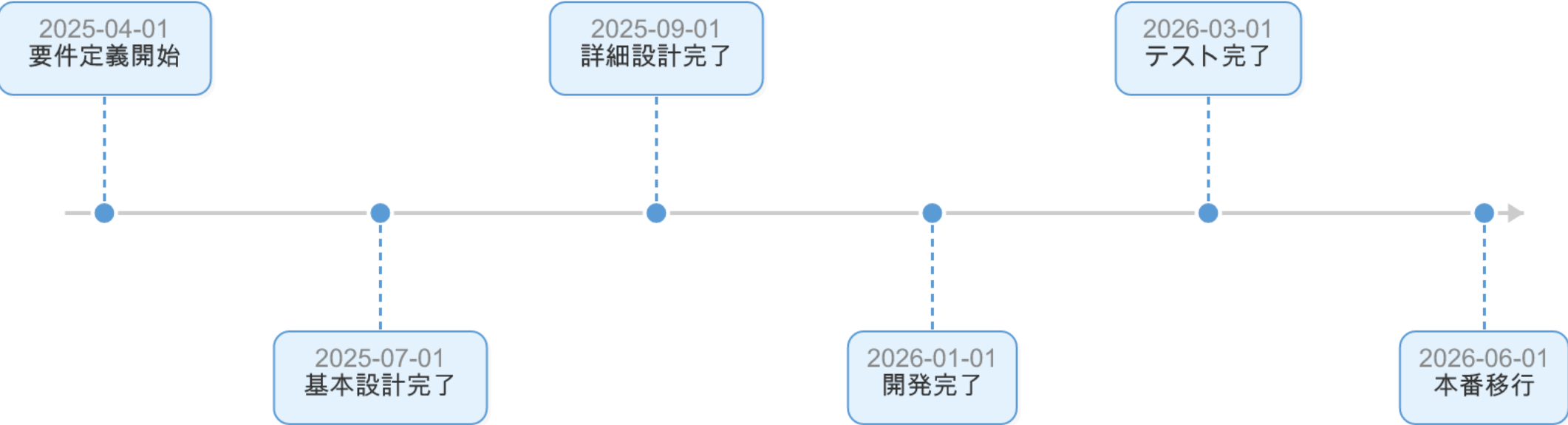
- 言語: Java
- 実績: 豊富
- 学習コスト: 中
- エコシステム: 大規模
- 保守性: 高い

Ruby on Rails

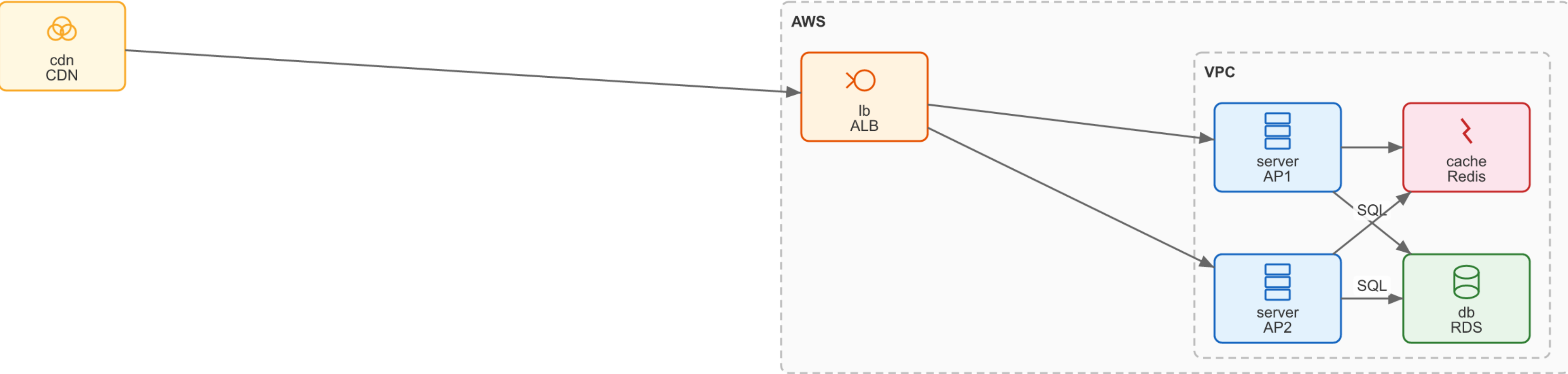
- 言語: Ruby
- 実績: 豊富
- 学習コスト: 低
- エコシステム: 中規模
- 保守性: 中

Next.js

- 言語: TypeScript
- 実績: 増加中
- 学習コスト: 中
- エコシステム: 大規模
- 保守性: 高い







プロジェクト SWOT

S - Strengths	W - Weaknesses
- チームの技術力が高い - 既存業務知識が豊富	- レガシーシステムとの依存 - テスト環境が不十分
O - Opportunities	T - Threats
- クラウド移行でコスト削減 - DXによる業務効率化	- 納期遅延リスク - 要件変更の頻発



(おまけ) このスライドの正体

このスライド自体が、ただの Markdown テキストから自動生成されています。



```
1  # MDD — Markdown with Diagrams
2
3  ドキュメントに図を入れたい。でも、もっと簡単に。
4
5  # 図の生成はすごく便利
6
7  ```usecase
8  actor 顧客
9  actor 管理者
10 package "認証" {
11   usecase ログイン
12 }
13 顧客 -> ログイン
```